

1. Introduction

Credit risk assessment is one of the most critical functions in modern banking and financial services. The ability to accurately predict whether a borrower will repay a loan in full has profound implications for the profitability, stability, and regulatory compliance of lending institutions. Historically, loan decisions were guided by manual underwriting processes relying on credit scores, income verification, and subjective judgment. However, with the exponential growth of financial data and advances in machine learning, automated risk scoring models have become the new standard.

Loan default events cause significant financial losses for banks, peer-to-peer (P2P) lending platforms, and other financial intermediaries. In a P2P lending context, where retail investors fund individual loans, the consequences of a high default rate are particularly acute, as investors bear the credit risk directly. Accurately distinguishing high-risk borrowers from creditworthy ones is therefore essential to protect investor capital and maintain platform sustainability.

This research applies to supervised machine learning, specifically Logistic Regression, to build a binary classification model capable of predicting loan default. The study utilizes a publicly available loan dataset from Kaggle, originally derived from Lending Club, a leading U.S. P2P lending platform. The model is trained on historical borrower attributes such as FICO score, debt-to-income ratio, interest rate, and credit history to predict the probability that a given loan will not be fully repaid.

The report is structured to provide a comprehensive view of the research: from dataset description and preprocessing, through model building and evaluation, to practical findings and recommendations for financial institutions considering deploying machine learning in their credit decisioning pipelines.

2. Objectives Of the Study

The primary objectives of this research are as follows:

- To build a machine learning classification model capable of predicting loan default (binary outcome: fully paid vs. not fully paid) based on borrower financial attributes.
- To identify the most predictive features associated with loan default risk, such as FICO score, interest rate, and debt-to-income ratio.
- To evaluate model performance using standard classification metrics including Accuracy, Precision, Recall, F1-Score, and the ROC-AUC score.
- To handle the class imbalance inherent in default prediction datasets through SMOTE
- To categorize borrowers into interpretable risk tiers (Low, Medium, High) based on predicted default probability, enabling actionable risk management decisions.
- To demonstrate the practical applicability of machine learning for financial institutions seeking to automate and improve credit risk decisioning processes.

3. Dataset Overview

3.1. Dataset Source

The dataset used in this study was obtained from Kaggle, one of the world's largest platforms for data science competitions and public datasets. The specific dataset, titled "Loan Data", originates from LendingClub, the U.S.-based peer-to-peer lending company. LendingClub was one of the first online lending marketplaces, and its historical loan data is widely used in academic and industry research on credit risk modeling.

3.2. Dataset

1	credit.policy	purpose	int.rate	installment	log.annual.inc	dti	fico	days.with.cr.line	revol.bal	revol.util	inq.last.6mths	delinq.2yrs	pub.rec	not.fully.paid
2	1	debt_consolidation	0.1189	829.1	11.35040654	19.48	737	5639.958333	28854	52.1	0	0	0	0
3	1	credit_card	0.1071	228.22	11.08214255	14.29	707	2760	33623	76.7	0	0	0	0
4	1	debt_consolidation	0.1357	366.86	10.37349118	11.63	682	4710	3511	25.6	1	0	0	0
5	1	debt_consolidation	0.1008	162.34	11.35040654	8.1	712	2699.958333	33667	73.2	1	0	0	0
6	1	credit_card	0.1426	102.92	11.29973224	14.97	667	4066	4740	39.5	0	1	0	0
7	1	credit_card	0.0788	125.13	11.90496755	16.98	727	6120.041667	50807	51	0	0	0	0
8	1	debt_consolidation	0.1496	194.02	10.71441777	4	667	3180.041667	3839	76.8	0	0	1	1
9	1	all_other	0.1114	131.22	11.00209984	11.08	722	5116	24220	68.6	0	0	0	1
10	1	home_improvement	0.1134	87.19	11.40756495	17.25	682	3989	69909	51.1	1	0	0	0
11	1	debt_consolidation	0.1221	84.12	10.20359214	10	707	2730.041667	5630	23	1	0	0	0
12	1	debt_consolidation	0.1347	360.43	10.4341158	22.09	677	6713.041667	13846	71	2	0	1	0
13	1	debt_consolidation	0.1324	253.58	11.83500896	9.16	662	4298	5122	18.2	2	1	0	0
14	1	debt_consolidation	0.0859	316.11	10.93310697	15.49	767	6519.958333	6068	16.7	0	0	0	0
15	1	small_business	0.0714	92.82	11.51292546	6.5	747	4384	3021	4.8	0	1	0	0
16	1	debt_consolidation	0.0863	209.54	9.487972109	9.73	727	1559.958333	6282	44.6	0	0	0	0
17	1	major_purchase	0.1103	327.53	10.73891524	13.04	702	8159.958333	5394	53.4	1	0	0	0
18	1	all_other	0.1317	77.69	10.52277288	2.26	672	3895.958333	2211	88.4	0	0	0	0
19	1	credit_card	0.0894	476.58	11.60823564	7.07	797	6510.958333	7586	52.7	1	0	0	0
20	1	debt_consolidation	0.1039	584.12	10.49127422	3.8	712	2760	8311	59.8	0	0	0	0
21	1	major_purchase	0.1513	173.65	11.00209984	2.74	667	1126.958333	591	84.4	3	0	0	0
22	1	all_other	0.08	188.02	11.22524339	16.08	772	4888.958333	29797	23.2	1	0	0	0
23	1	all_other	0.0863	474.42	10.81977828	2.59	797	11951	5656	27.6	0	0	0	0
24	1	credit_card	0.1355	339.6	11.51292546	7.94	662	1939.958333	21162	57.7	0	0	0	0
25	1	credit_card	0.0788	484.85	11.73606902	7.05	782	5640.041667	16931	34.6	1	0	0	0
26	1	debt_consolidation	0.1229	320.19	11.26446411	8.8	672	3760.958333	4822	58.1	0	0	1	0
27	1	all_other	0.0901	159.03	12.4292162	10	712	1553.958333	14354	36.6	0	2	0	0
28	1	all_other	0.0743	155.38	11.08214255	0.28	802	4649.958333	1576	5.7	1	0	0	0
29	1	debt_consolidation	0.1375	255.43	9.998797732	14.29	662	1318.958333	4175	51.5	0	1	0	0
30	1	all_other	0.0743	155.38	12.20607265	0.28	772	4516.958333	3164	13.7	0	0	0	0
31	1	all_other	0.0743	155.38	12.20607265	3.72	812	6778.958333	85607	0.7	0	0	0	0
32	1	debt_consolidation	0.0807	156.84	11.51292546	2.3	742	3148.958333	9698	19.4	0	0	0	0
33	1	credit_card	0.1028	275.38	9.798127037	6.4	692	7469.958333	8847	26.9	1	1	0	0
34	1	home_improvement	0.0743	155.38	11.91839057	0	777	7128.958333	6053	19.5	0	0	0	0
35	1	home_improvement	0.0807	78.42	11.60823564	11.33	762	6059.958333	7274	13.1	0	0	0	0
36	1	credit_card	0.087	158.3	11.22524339	15.55	757	4770	66033	23	0	0	0	0
37	1	all_other	0.0743	155.38	11.46163217	3.83	772	2535	3660	6.8	0	0	0	0
38	1	credit_card	0.0743	155.38	11.35040654	0.31	787	3540	216	0.6	0	0	0	0
39	1	credit_card	0.1122	164.23	11.28978191	1.21	672	3990	27185	16.1	3	0	1	0
40	1	debt_consolidation	0.0996	161.25	10.77895629	8.03	707	4320	28329	48.6	1	0	0	0
41	1	home_improvement	0.0933	159.77	12.10071213	11.93	717	4498.958333	60568	39.2	1	0	0	0
42	1	home_improvement	0.0807	156.84	12.10071213	5.55	742	4019	40934	26.3	0	0	0	0
43	1	debt_consolidation	0.0743	155.38	11.69524702	2.29	777	3540	8379	16.9	0	0	0	0
44	1	small_business	0.087	94.98	10.1266311	14.54	752	2460	3660	7.8	1	0	0	0
45	1	home_improvement	0.1028	97.2	12.20607265	0	677	3779.958333	0	0	0	1	0	0
46	1	credit_card	0.1122	164.23	10.30895266	18.64	702	5190	15840	47.1	0	0	0	0
47	1	credit_card	0.0838	167.02	11.21182037	14.37	797	5610.958333	6844	14.4	0	0	0	0
48	1	educational	0.0775	168.6	8.987196821	3	762	6659.958333	1321	16.5	0	0	0	0
49	1	all_other	0.0838	160.72	10.37349118	14.78	772	4379	4737	23.9	0	0	0	0
50	1	credit_card	0.0901	159.03	11.19821472	9.96	722	3089.958333	23489	37.6	0	0	0	0

3.2. Features Overview

Features	Type	Description
Credit.policy	Integer (Binary)	1 if the borrower met Lending Club's credit underwriting criteria; 0 otherwise
purpose	Categorical (String)	Stated purpose of the loan (e.g., debt consolidation, credit card, education)
Int.rate	Float	Loan interest rate (annual); higher rates indicate higher perceived risk
installment	Float	Monthly repayment installment amount in USD
log.annual.inc	Float	Natural log of the borrower's annual income
dti	Float	Debt-to-income ratio; total debt obligations divided by gross income
fico	Integer	FICO credit score of the borrower at time of loan application
days.with.cr.line	Float	Number of days the borrower has had a credit line
revol.bal	Integer	Revolving credit balance outstanding
revol.util	Float	Revolving line utilization rate (% of credit used)
Inq.last.6mths	Integer	Number of credit inquiries in the last 6 months
delinq.2yrs	Integer	Number of 30+ day past-due payment delinquencies in last 2 years
pub.rec	Integer	Number of derogatory public records (bankruptcies, tax liens, judgments)
not.fully.paid	Integer (Binary)	1 indicates default; 0 otherwise

3.3. Dataset Dimensions

Attribute	Value
Total Records (Rows)	9,578
Total Features (Columns)	14 (including target)
Input Features	13
Target Variable	not.fully.paid (Binary: 0 or 1)
Missing Values	None
Data Type	Mix of Integer, Float, and Categorical (string)

4. Model Building

4.1. Importing Necessary libraries

a) Code 1:

```
# Importing libraries
import pandas as pd
import numpy as np
# Visualization
import matplotlib.pyplot as plt
import seaborn as sns
#SMOTE
from imblearn.over_sampling import SMOTE
# ML libraries
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
# Evaluation
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score,
roc_auc_score
```

b) Explanation:

To begin the analysis and model development, the necessary Python libraries were imported. These libraries provide the foundational tools required for data manipulation, visualization, modeling, and evaluation. The imported modules can be categorized into the following key areas:

- **Data Manipulation and Numerical Operations:**
 - **Pandas:** Utilized for loading, cleaning, and manipulating structured data using DataFrames.
 - **NumPy:** Employed for efficient numerical computations and handling multidimensional arrays.
- **Data Visualization:**
 - **Matplotlib.Pyplot & Seaborn:** Used to create statistical graphics and visual plots, which are essential for Displaying Confusion Matrix.
- **Handling Imbalanced Data:**
 - **SMOTE (from imblearn):** The Synthetic Minority Over-Sampling Technique is imported to address potential class imbalances in the target variable by generating synthetic samples for the minority class.
- **Machine Learning Preprocessing and Modeling:**
 - **train_test_split:** Used to partition the dataset into training and testing subsets, ensuring the model can be evaluated on unseen data.
 - **StandardScaler:** Applied to standardize the features by removing the mean and scaling to unit variance, which is crucial for distance-based and linear models.
 - **LogisticRegression & RandomForestClassifier:** The core predictive models selected for this classification task. Logistic regression provides a linear baseline, while Random Forest offers a robust ensemble approach.
- **Model Evaluation Metrics:**
 - **classification_report, confusion_matrix, accuracy_score, & roc_auc_score:** Imported from sklearn.metrics to rigorously assess model performance. These metrics provide a comprehensive view of the models' predictive accuracy, precision, recall, and ability to distinguish between classes.

4.2. Loading Dataset

a) Code 2:

```
# Load dataset
df = pd.read_csv("loan_data.csv")
# Checking the data info
df.head()
df.info()
# Checking the target distribution
df['not.fully.paid'].value_counts()
```

b) Output:

```
RangeIndex: 9578 entries, 0 to 9577
Data columns (total 14 columns):
#   Column              Non-Null Count  Dtype
---  -
0   credit.policy        9578 non-null   int64
1   purpose              9578 non-null   str
2   int.rate             9578 non-null   float64
3   installment          9578 non-null   float64
4   log.annual.inc       9578 non-null   float64
5   dti                  9578 non-null   float64
6   fico                 9578 non-null   int64
7   days.with.cr.line    9578 non-null   float64
8   revol.bal            9578 non-null   int64
9   revol.util           9578 non-null   float64
10  inq.last.6mths       9578 non-null   int64
11  delinq.2yrs          9578 non-null   int64
12  pub.rec              9578 non-null   int64
13  not.fully.paid       9578 non-null   int64
dtypes: float64(6), int64(7), str(1)
memory usage: 1.0 MB

not.fully.paid
0      8045
1     1533
Name: count, dtype: int64
```

c) Explanation:

Following the environment setup, the dataset (loan_data.csv) was loaded into a Pandas DataFrame to conduct an initial exploratory data analysis (EDA) and assess the overall data structure.

- **Dataset Overview:** The dataset consists of 9,578 records and 14 distinct features. The `df.info()` output confirms that the dataset is highly complete, with absolutely no missing or null values across any of the columns. The features represent a mix of data types:
- **Numerical Features:** 13 variables are numerical (a mix of 6 float64 and 7 int64 types), representing key financial and credit metrics such as interest rate (`int.rate`), FICO score (`fico`), and revolving balance (`revol.bal`).
- **Categorical Features:** 1 variable (`purpose`) is stored as a string, representing the stated reason for the loan. This will require categorical encoding before model training.
- **Target Variable Distribution & Class Imbalance:** The primary objective of the model is to predict the `not.fully.paid` status. An analysis of this target variable revealed a significant class imbalance:

Class 0 (Fully Paid): 8,045 instances (~84%)

Class 1 (Not Fully Paid / Defaulted): 1,533 instances (~16%)

4.3. Data Preprocessing & Feature Engineering

a) Code 3:

```
Convert 'purpose' column into dummy variables
df = pd.get_dummies(df, columns=['purpose'], drop_first=True)
Defining Features (X) and target (y)
X = df.drop('not.fully.paid', axis=1)
y = df['not.fully.paid']
```

b) Explanation:

Before training the machine learning models, the dataset required specific transformations to ensure the data was properly structured and compatible with the mathematical requirements of the chosen algorithms.

- **Categorical Encoding:**
Most machine learning models cannot directly interpret text or string data; they require numerical inputs. The exploratory analysis identified “purpose” as a categorical string variable. To resolve this, one-hot encoding was applied using the `pd.get_dummies()` function. This transformation converted the single purpose column into several separate binary columns (containing 0s and 1s), each representing a specific loan purpose category. Crucially, the `drop_first=True` argument was applied. This mathematically necessary step removes the first dummy column to prevent multicollinearity (often referred to as the “dummy variable trap”). Removing this baseline reference group ensures the structural integrity of the dataset, which is especially important for linear models like Logistic Regression.
- **Feature and Target Separation:**
Following the encoding process, the dataset was formally partitioned into its independent and dependent variables, establishing the standard architecture for supervised learning:
 - **Feature Matrix (X):** Created by dropping the target column (`not.fully.paid`). This matrix now contains all the independent variables—both the original numerical features and the newly created dummy variables—that the model will analyze to identify risk patterns.
 - **Target Vector (y):** Isolated entirely as the `not.fully.paid` column. This represents the dependent variable (the ground-truth labels) that the model will be trained to predict.

4.4. Data Splitting, Scaling & Handling Imbalance

a) Code 4:

```
#Splitting dataset into train and test
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.2, random_state=42 )
#Scale numerical features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
#Apply SMOTE ONLY on training data
sm = SMOTE(random_state=42)
X_train_res, y_train_res = sm.fit_resample(X_train, y_train)
```

b) Explanation:

With the dataset preprocessed, the next phase involved preparing the data specifically for the model training pipeline. This phase required three critical steps to ensure the model would learn effectively and be evaluated fairly.

- **Train-Test Data Splitting:**

To evaluate the model's ability to generalize to unseen, future data, the dataset was partitioned using `train_test_split`.

- 80% of the data (`X_train`, `y_train`) was allocated for training the model.
- 20% of the data (`X_test`, `y_test`) was held back as a pristine testing set.
- A `random_state` was defined to ensure this exact split can be reproduced in future runs of the analysis.

- **Feature Scaling (Standardization):**

Machine learning algorithms often perform poorly when numerical features exist on drastically different scales (e.g., comparing a FICO score in the hundreds to an annual income in the tens of thousands). To resolve this, a `StandardScaler` was utilized.

- **Preventing Data Leakage:** Crucially, the scaler was fitted only to the training data (`fit_transform`). The testing data was only transformed (`transform`) using the parameters learned from the training set. This strictly prevents "data leakage," ensuring the model does not inadvertently gain information about the test set during the training phase.

- **Addressing Class Imbalance with SMOTE:**

As identified during the exploratory data analysis, the target variable (`not.fully.paid`) was heavily skewed toward fully paid loans. To correct this, the Synthetic Minority Over-sampling Technique (SMOTE) was applied.

- **Strategic Application:** SMOTE was explicitly applied only to the training data (`X_train`, `y_train`), generating synthetic examples of the minority class (defaulted loans) until both classes were equally represented in a new balanced training set (`X_train_res`, `y_train_res`). Applying SMOTE solely to the training set ensures the model learns to identify defaults accurately, while the testing set remains untouched and fully representative of real-world, imbalanced conditions for an honest evaluation.

4.5. Logistic Regression Model Training and Prediction

a) Code 5:

```
# Building and training the LR model with class balancing
model = LogisticRegression(max_iter=1000)
model.fit(X_train_res, y_train_res)
# Predict class and probability
y_pred_lr = model.predict(X_test)
y_prob_lr = model.predict_proba(X_test)[:,1]
```

b) Explanation:

Following the preparation of a balanced and scaled training dataset, the analysis transitions into the formal modeling phase. For this stage, a Logistic Regression classifier was implemented as the foundational predictive model.

- **Model Initialization and Training:**
 - **Algorithm Selection (LogisticRegression):** Logistic regression was chosen as a strong baseline model due to its interpretability and efficiency in binary classification tasks.
 - **Convergence Tuning (max_iter=1000):** The max_iter parameter was explicitly increased to 1,000 (above the standard default). This ensures that the algorithm's optimization solver has an adequate number of iterations to converge on the optimal weights for the features, preventing premature termination during training.
 - **Model Fitting (model.fit):** The model was trained using the X_train_res and y_train_res datasets. Crucially, this means the model learned the underlying financial patterns from the SMOTE-balanced data, allowing it to appropriately weigh the characteristics of both "Fully Paid" and "Defaulted" loans without bias toward the majority class.
- **Generating Predictions:**

Once trained, the model's predictive capabilities were tested against the pristine, unseen testing data (X_test). Two distinct types of predictions were generated to facilitate a comprehensive evaluation:

 - **Binary Class Predictions (y_pred_lr):** Using the .predict() method, the model assigned a strict categorical label (0 for Fully Paid, 1 for Not Fully Paid) to each loan application in the test set based on a standard 0.5 probability threshold.
 - **Probability Scores (y_prob_lr):** Using the .predict_proba() method, the model calculated the precise probability that a given loan belongs to each class. By applying the [:,1] slice, the code deliberately isolates the probability of the positive class (the likelihood of default). Extracting these continuous probability scores is a critical step, as it enables the calculation of advanced performance metrics like the ROC-AUC score and allows for future threshold adjustments based on business risk tolerance.

4.6. Random Forest Model Training and Prediction

a) Code 6:

```
# Building and training the RF model
rf = RandomForestClassifier(n_estimators=100,random_state=42)
rf.fit(X_train_res, y_train_res)
#Predicting Class and Probability with RF
y_pred_rf = rf.predict(X_test)
y_prob_rf = rf.predict_proba(X_test)[:,:1]
```

b) Explanation:

To establish a robust comparison against the linear baseline established by the Logistic Regression model, a Random Forest Classifier was implemented. This non-linear, ensemble machine learning algorithm is highly effective at capturing complex interactions between financial features without requiring extensive parameter tuning.

- **Model Initialization and Training:**
 - **Algorithm Selection (RandomForestClassifier):** Random Forest operates by constructing a multitude of individual decision trees during the training phase and outputting the mode of the classes (for classification) of the individual trees. This ensemble approach inherently reduces the risk of overfitting compared to a single decision tree.
 - **Hyperparameter Configuration (n_estimators=100):** The model was configured to build 100 distinct decision trees (n_estimators). This is a standard and robust baseline that provides a strong balance between computational efficiency and predictive accuracy. A random_state of 42 was also set to ensure the specific architecture of the forest is fully reproducible.
 - **Model Fitting (rf.fit):** Consistent with the previous modeling step, the Random Forest was trained using the SMOTE-balanced dataset (X_train_res, y_train_res). This ensures the ensemble algorithm evaluates an equal representation of "Fully Paid" and "Defaulted" loans, preventing the individual trees from developing a bias toward the majority class.
- **Generating Predictions:**

Following training, the Random Forest model was evaluated against the unseen testing set (X_test), generating the same dual-output structure as the baseline model:

 - **Binary Class Predictions (y_pred_rf):** The .predict() method was used to generate definitive binary classifications (0 or 1) for each loan in the test set, representing the ensemble's final "vote" on whether the loan will be fully paid or will default.
 - **Probability Scores (y_prob_rf):** The .predict_proba() method was utilized to extract the continuous probability scores, specifically isolating the likelihood of the positive class (default) using the[:, 1] slice. Capturing these probabilities from the Random Forest is essential for generating comparative evaluation metrics (such as ROC curves) to determine which algorithm offers superior risk stratification.

5. Evaluation Results

5.1. Logistic Regression Model

a) Code 7:

```
# Evaluation Metrics (LR Model)
print('Logistic Regression : \n')
# Accuracy
accuracy = accuracy_score(y_test, y_pred_lr)
print("Accuracy:", accuracy)
# Classification report
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred_lr))
# ROC-AUC score
roc = roc_auc_score(y_test, y_prob_lr)
print("\nROC-AUC Score:", roc)
# Confusion matrix
cm = confusion_matrix(y_test, y_pred_lr)
plt.figure(figsize=(6,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix Heatmap (LR)")
plt.show()
```

b) Output:

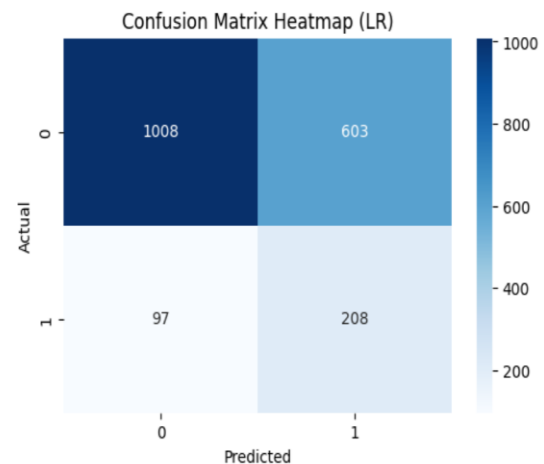
Logistic Regression :

Accuracy: 0.6346555323590815

Classification Report:

	precision	recall	f1-score	support
0	0.91	0.63	0.74	1611
1	0.26	0.68	0.37	305
accuracy			0.63	1916
macro avg	0.58	0.65	0.56	1916
weighted avg	0.81	0.63	0.68	1916

ROC-AUC Score: 0.7084368735435684



c) Explanation:

With the Logistic Regression baseline model trained, its performance was rigorously evaluated against the unseen testing set (X_{test}). The evaluation utilized a combination of accuracy, classification metrics, the ROC-AUC score, and a confusion matrix to provide a holistic view of the model's predictive behavior, particularly in the context of the SMOTE-balanced training data.

- **Overall Predictive Power:**
 - **Accuracy (63.5%):** The model achieved an overall accuracy of approximately 63.5%. While this metric provides a high-level summary, it is often misleading in credit risk scenarios.
 - **ROC-AUC Score (0.71):** To better assess the model's underlying ability to distinguish between fully paid and defaulted loans independent of a strict threshold, the Receiver Operating Characteristic - Area Under the Curve (ROC-AUC) score was calculated. A score of 0.708 indicates acceptable discrimination ability, demonstrating that the model performs significantly better than random guessing (0.50).
- **Confusion Matrix Breakdown:**

The confusion matrix heatmap visualizes exactly where the model succeeded and where it made errors across the 1,916 test cases:

 - **True Negatives (1,008):** The model correctly identified 1,008 loans that were fully paid.
 - **False Positives (603):** The model incorrectly flagged 603 good loans as potential defaults (Type I error).
 - **True Positives (208):** The model successfully caught 208 actual defaults.
 - **False Negatives (97):** The model missed 97 defaults, incorrectly predicting they would be paid off (Type II error).
- **Business Context & The Impact of SMOTE:**

The detailed classification report reveals the direct impact of the SMOTE technique applied during preprocessing.

In a financial context, the cost of a False Negative (approving a loan that eventually defaults) is substantially higher than a False Positive (rejecting a loan that would have been paid back). Because we balanced the training data, the model was forced to become highly sensitive to the minority class (Defaults/Class 1).

 - **Strong Risk Capture (Recall):** The model achieved a Recall of 0.68 for Class 1, meaning it successfully identified 68% of all actual defaults in the testing set.

- **The Precision Trade-off:** The trade-off for this high sensitivity is a low Precision of 0.26 for Class 1. Because the model casts a wide net to catch defaults, it generates a high number of False Positives (603).

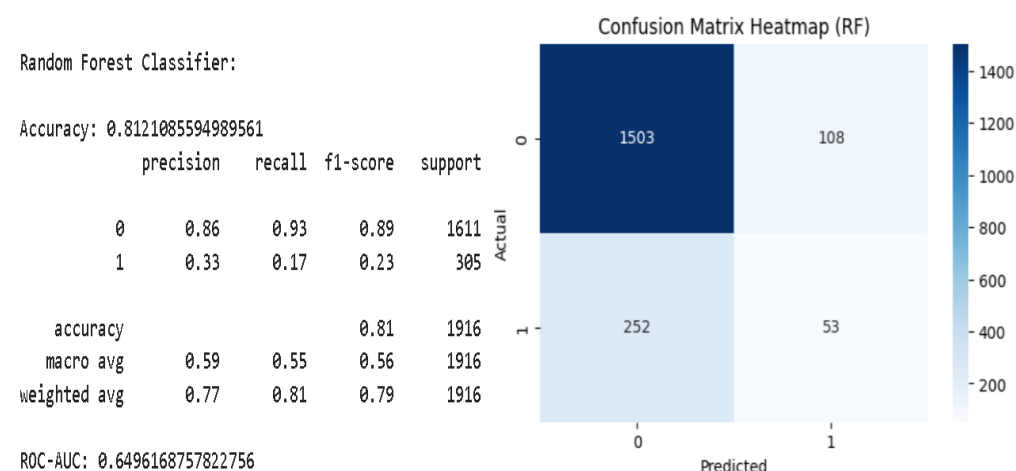
Ultimately, this evaluation shows a conservative baseline model. It successfully mitigates extreme financial risk by identifying the majority of bad loans, but it does so at the expense of turning away a significant portion of viable customers.

5.2. Random Forest Model

a) Code 8:

```
# Evaluation Metrics (RF Model)
print('Random Forest Classifier:\n')
# Accuracy
accuracy = accuracy_score(y_test, y_pred_rf)
print("Accuracy:", accuracy)
# Classification Report
print(classification_report(y_test, y_pred_rf))
# ROC-AUC Score
print("ROC-AUC:", roc_auc_score(y_test, y_prob_rf))
# Confusion Matrix
cm = confusion_matrix(y_test, y_pred_rf)
plt.figure(figsize=(6,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix Heatmap (RF)")
plt.show()
```

b) Output:



c) Explanation:

Following the baseline model, the Random Forest Classifier was evaluated against the same testing set (X_{test}). The results reveal a stark contrast in predictive behavior compared to the Logistic Regression model, highlighting the importance of looking beyond basic accuracy metrics in imbalanced classification tasks.

- **Overall Predictive Power:**
 - **Accuracy (81.2%):** At first glance, the Random Forest model appears significantly superior, achieving an overall accuracy of over 81% (compared to Logistic Regression's 63.5%).

- **ROC-AUC Score (0.65):** However, the ROC-AUC score tells a different story. Dropping to 0.649, this metric indicates that the ensemble model actually has less underlying ability to distinguish between good and bad loans across different probability thresholds than the simpler linear model.
- **Confusion Matrix Breakdown:** The confusion matrix highlights a shift in the types of errors the model is making across the 1,916 test cases:
 - **True Negatives (1,503):** The model correctly identified a massive majority of the fully paid loans.
 - **False Positives (108):** The model only incorrectly flagged 108 good loans as defaults (a significant drop in Type I errors).
 - **True Positives (53):** The model only successfully caught 53 actual defaults.
 - **False Negatives (252):** The model missed 252 defaults, incorrectly predicting they would be paid off (a massive spike in Type II errors).
- **Business Context & Comparative Analysis:**
The classification report exposes the fundamental flaw in this specific Random Forest iteration for this business use case. Despite being trained on SMOTE-balanced data, the model heavily favored the majority class.
 - **Poor Risk Capture (Recall):** The most critical metric for a credit risk model is its ability to identify actual defaults. The Random Forest achieved a dismal Recall of 0.17 for Class 1, meaning it missed 83% of all defaulting loans in the test set.
 - **Higher Precision:** Because the model was incredibly conservative about predicting a default, when it did predict one, it was slightly more accurate than the baseline (Precision of 0.33 vs 0.26). This resulted in far fewer False Positives (turning away fewer good customers).

5.3. Defining Risk Categories Based on Probabilities

a) Code 9:

```
# Create risk categories based on probability
def risk_category(p):
    if p < 0.3:
        return "Low Risk"
    elif p < 0.7:
        return "Medium Risk"
    else:
        return "High Risk"
# Create dataframe
risk_df = pd.DataFrame({ 'Default_Probability': y_prob_lr })
risk_df['Risk_Category'] = risk_df['Default_Probability'].apply(risk_category)
risk_df.head()
```

b) Output:

	Default_Probability	Risk_Category
0	0.742774	High Risk
1	0.459436	Medium Risk
2	0.439539	Medium Risk
3	0.552179	Medium Risk
4	0.335117	Medium Risk

c) Explanation:

To translate the raw statistical outputs of the Logistic Regression model into actionable business intelligence, a risk stratification framework was implemented. This step bridges the gap between machine learning probabilities and practical decision-making for loan officers.

- **Defining the Risk Framework:** A custom Python function (`risk_category`) was created to segment the continuous probability scores (ranging from 0 to 1) into three distinct, easily interpretable risk tiers. The thresholds were defined as follows:
 - **Low Risk:** Probability of default strictly less than 0.30 ($< 30\%$). These represent the safest loan applicants.
 - **Medium Risk:** Probability of default between 0.30 and 0.69. These applications require closer manual review or potentially adjusted loan terms (e.g., higher interest rates).
 - **High Risk:** Probability of default of 0.70 or greater ($\geq 70\%$). These represent applications with a severe likelihood of default, which would typically be rejected under standard policy.
- **Application to Model Outputs:** A new analytical DataFrame (`risk_df`) was constructed specifically to house these insights. The continuous probability scores generated by the baseline Logistic Regression model (`y_prob_lr`) were loaded into this structure. The `apply()` method was then utilized to map the predefined `risk_category` logic across every individual probability score.

6. Conclusion

This project successfully developed and evaluated a machine learning pipeline designed to predict loan defaults and mitigate financial risk. By addressing the inherent class imbalance in the dataset using the SMOTE technique, the models were forced to prioritize the identification of high-risk applicants, yielding highly insightful comparative results.

- **Key Findings:**

The evaluation of the two distinct algorithms highlighted a fundamental trade-off in credit risk modeling:

- **The Random Forest Classifier** achieved a much *higher overall accuracy (81.2%)*, but this metric was deceptive. It heavily favored the majority class, resulting in a dangerously *low Recall (17%) for defaults*. In a business context, this model would approve far too many bad loans, exposing the institution to significant financial loss.
- **The Logistic Regression Model** proved to be the superior choice for risk mitigation. While its overall *accuracy was lower (63.5%)*, it achieved a robust *ROC-AUC score of 0.71* and successfully *captured 68% of all actual defaults (high Recall)*. It acted as a conservative gatekeeper, prioritizing the prevention of financial loss over maximizing approval rates.
- **Operational Impact and Recommendations:**

Because minimizing False Negatives (approving a loan that will default) is the primary objective in credit risk management, the Logistic Regression model is the recommended foundational algorithm for this specific use case.

To operationalize these findings, the continuous probability outputs of the Logistic Regression model were transformed into a Risk Stratification Framework (Low, Medium, and High Risk tiers). This framework allows the business to transition from binary "approve/reject" decisions to nuanced, risk-based pricing strategies.